

03_m5_m6b

M5 - Semantic Layer live [Wk1-MVP]

The registry `models/semantic/semantic_layer.yaml` (registry v2.0.0, 47 metrics / 19 dimensions, descriptive field names `metric_name / sql_definition / aggregation_method / dimensions_supported`) **already exists and is RI-clean**. Claude Code's job is NOT to author metrics. It is to (1) wrap the registry in a thin dbt exposure (`metrics__schema.yaml`) so it joins the dbt DAG, and (2) write `scripts/validate_semantic.py` , the standalone compile gate that proves every `numerator / denominator` , every derived `expr {token}` , every `dimensions_supported` entry, every `agents[]` entry, and every `grain` resolves against the live registry. The validator is the M5 exit gate (METRIC_GOVERNANCE_GUIDE.md §6.1; invariant I1).

Filename-drift to fix in this milestone. Two files sit on disk: `semantic_layer.yaml` (v2.0.0, 47 metrics — **canonical**) and the retired `semantic_models.yaml` (v1, ~28 metrics). MCP_ARCHITECTURE.md §3 and CLAUDE.md §5 still cite `semantic_models.yaml` . Point the validator (and later `mcp_servers.yaml`) at `semantic_layer.yaml` . Have Claude confirm which file before writing a line — if it validates the v1 file, the gate is green but governs the wrong layer.

Context to load

- CLAUDE.md (auto-loads at repo root — pins canonical names, `reached_*` , `SUM(num)/SUM(den)` , the 10 channel groups; D1).
- `@docs/METRIC_GOVERNANCE_GUIDE.md` — sections §2 (per-metric field contract), §2.2 (type→population matrix), §6.1 (referential-integrity compile = exactly what the validator must enforce), §9.1 (canonical dimension list, 6 valid agents).
- `@models/semantic/semantic_layer.yaml` — the live registry. Open it so Claude reads the **actual field names** (`metric_name` , `sql_definition` , `dimensions_supported` , `agents`) and the real metric set, not what it remembers (D2/D3).
- The milestone tracker / `/resume` output for the next-step pin (D8: fresh session, M5 only).

Files to provide

File	Why
<code>@models/semantic/semantic_layer.yaml</code>	Source of truth — Claude reads its keys to write field-accurate validation; never regenerated.

File	Why
@docs/METRIC_GOVERNANCE_GUIDE.md (§2, §2.2, §6.1, §9.1)	The spec the validator encodes; cite per check.
@models/semantic/metrics__schema.yml	Once it exists, attach for the exposure edit.
@scripts/validate_semantic.py	Once it exists, attach for iteration.

Prompts to use

Plan first (D9):

```
You are implementing Helios milestone M5. Plan mode only – do NOT write code yet.
The registry @models/semantic/semantic_layer.yaml ALREADY EXISTS (registry v2.0.0, 47 metrics).
Do NOT regenerate or edit it. First: open it and tell me the EXACT top-level keys and the EXACT
per-metric field names it uses (metric_name? name? sql_definition? sql? dimensions_supported? agents
and confirm it is the v2.0.0 file (not the retired semantic_models.yaml). Then propose:
(1) models/semantic/metrics__schema.yml – a dbt exposure depending on the five mart grains, and
(2) scripts/validate_semantic.py – a standalone, no-network, no-warehouse validator enforcing the
METRIC_GOVERNANCE_GUIDE §6.1 compile (ratio num/den resolve, derived expr {token}s resolve,
dimensions_supported all defined, agents[] in {Monitor,Decompose,Diagnose,Prescribe,Narrator,Cr
grain in grains:, and the §2.2 type→population matrix).
Cite the §6.1 bullet each check implements. Show me the plan before any edit.
```

Then implement against the confirmed field names:

```
[ROLE] Implement Helios M5, file scripts/validate_semantic.py.
[SOURCE OF TRUTH] Encode the METRIC_GOVERNANCE_GUIDE §6.1 referential-integrity compile and the §2.2
type→population matrix. Follow CLAUDE.md conventions (auto-loaded).
[UPSTREAM] Read field/metric names ONLY from @models/semantic/semantic_layer.yaml – use the EXACT fi
names you reported in the plan (the v2 file uses descriptive names: metric_name, sql_definition,
aggregation_method, dimensions_supported, agents, grain).
[TASK] Pure-Python validator: load the YAML, collect metric_names/dimension_names/grain keys, then a
(1) every ratio numerator+denominator is a defined metric_name; (2) every derived expr {token}
(regex \{([a-zA-Z0-9_+])\}) is a defined metric_name; (3) every dimensions_supported entry is a def
dimension; (4) every agents[] entry ∈ {Monitor,Decompose,Diagnose,Prescribe,Narrator,Critic};
(5) every grain ∈ grains:; (6) §2.2 matrix (count/sum→sql_definition set, num/den/expr null;
ratio→num+den set, expr null; derived→expr set, num/den null). Accumulate ALL errors; on any error
print "FAIL: <n> dangling/invalid reference(s)" naming each offender and exit 1; else print
"PASS: <m> metrics, <d> dimensions – 0 dangling references" and exit 0. No network, no warehouse, n
[TEST-FIRST] Before the validator, write tests/test_validate_semantic.py with: a PASS case on the re
registry, and a NEGATIVE case that copies the registry to a temp file, corrupts one ratio denominat
to 'sessionz', and asserts exit 1 + the offending name in output.
[CONSTRAINTS] Point REGISTRY at models/semantic/semantic_layer.yaml (NOT semantic_models.yaml). Do NO
invent metric/dimension names; read them from the file. If a field name is not what you expected, S
```

```
[OUTPUT] Run `python scripts/validate_semantic.py` and `pytest tests/test_validate_semantic.py -v`; both outputs. Cite the §6.1 bullet each check implements.
```

Exposure wrapper (small, can share the same session):

```
[ROLE] Implement Helios M5, file models/semantic/metrics__schema.yml.
[SOURCE OF TRUTH] METRIC_GOVERNANCE_GUIDE §1 (single-source-of-truth) + CLAUDE.md §5 (semantic layer a view; the YAML is the governance object).
[TASK] A dbt v2 exposure named semantic_layer (type: application, maturity: high, owner analytics-en that depends_on ref('fct_funnel'), ref('fct_sessions'), ref('fct_orders'), ref('fct_order_items'), ref('fct_cohorts')). Description: it is the single source of truth for all 47 metrics / 19 dimensions that semantic-mcp.build_query reads, RI-enforced by scripts/validate_semantic.py.
[TEST] Run `dbt parse` and paste output – the exposure must register with no ref errors.
[OUTPUT] Cite the doc section.
```

Preventing hallucinations

- D2/D3 specialized: the registry is the upstream. Make Claude **read the real field names from semantic_layer.yaml** before coding — the v2 file uses `metric_name / sql_definition / dimensions_supported`, not `name / sql / dimensions`.
◀ If Claude codes against remembered keys it will silently match nothing and print a false PASS. ▶
- D5: forbid inventing metric names in `REQUIRED_METRICS`. If you include a floor of must-exist names, derive them from the open registry, not from memory — a non-existent "required" name turns the gate into a false FAIL.
- D6 specialized: the validator IS the machine that catches hallucinated refs. Treat any dangling `numerator / denominator / {token} / dimension / grain` as a **hard fail** — never "fix" it by editing `semantic_layer.yaml`; fix the typo or the validator. Editing a governed definition to satisfy a tool is backwards (the registry is RI-clean by construction).
- Extract `expr` tokens with the `\{ ... \}` regex, never a comma split: `SAFE_DIVIDE({revenue},{users})` has two tokens, both must resolve.
- Reject `Orchestrator` in any `agents[]` (it plans, consumes no metrics) — only the 6 in the valid set are legal (§9.1).

Reviewing generated code

- R1: each check cites the §6.1 bullet it implements.
- R3 (the load-bearing check): Claude RAN the validator AND the negative test. `python scripts/validate_semantic.py; echo exit=$? → PASS: 47 metrics, 19 dimensions – 0 dangling references, exit=0`. The negative test must print `FAIL` naming

denominator 'sessionz' is not a defined metric_name and exit 1. A gate that cannot fail is not a gate — reject any PR without the proven-FAIL test.

- R2: dbt parse succeeds; the semantic_layer exposure depends on the five grains with no ref error.
- R4: confirm REGISTRY path = semantic_layer.yaml (grep the script). If it loaded semantic_models.yaml, the metric count prints ~28/44 and the gate governs the retired layer — reject.
- R5: type→population matrix actually enforced (a ratio with a stray expr, or a derived with numerator set, is flagged even when names resolve).

M6 - Grounding MCP pair [Wk2-v1: build ONE server]

Full design = semantic-mcp (only path to SQL) + warehouse-mcp (sole BigQuery client, dry-run + byte-budget gate). **LEAN (Wk2-v1): build ONE server.** Pick semantic-mcp (build_query from the registry) — it carries the grounding story ("the model composes governed metrics, never writes raw SQL → 0 hallucinated columns") and is data-independent enough to demo without a live warehouse round-trip. (M6b covers the stats alternative.) The critical anti-hallucination property is **structural**: build_query interpolates only registry templates, so a column not in the registry has no surface to enter through (invariant I3). Connect the server to Claude Code so Claude can ONLY get data through the governed tool.

Context to load

- CLAUDE.md (auto: MCP server/tool table, grounding rules G1/G5).
- @docs/MCP_ARCHITECTURE.md — §6.2 (semantic-mcp tool I/O), §7 (registry binding = the anti-hallucination core), §8 (compile-time integrity, fail-load), §9 (the build_query resolver skeleton — copy/adapt), §5 (error taxonomy), §11 (semantic-mcp tests).
- @docs/METRIC_GOVERNANCE_GUIDE.md §3 (the authoring↔resolver field-name mapping — metric_name→name, sql_definition→sql, aggregation_method→agg, dimensions_supported→dimensions) — load-bearing: the v2 YAML is authored in descriptive names, the resolver reads short keys.
- @models/semantic/semantic_layer.yaml (the registry the server loads + RI-compiles at startup).
- The expected golden SQL shape from MCP_ARCHITECTURE §6.2 (the session_conversion_rate × device_category snapshot).

Files to provide

File	Why
@docs/MCP_ARCHITECTURE.md (§5, §6.2, §7, §8, §9, §11)	Skeletons + contract + tests; Claude copies/adapts, not invents.
@models/semantic/semantic_layer.yaml	The registry the resolver reads; real metric/dim/grain names.
@docs/METRIC_GOVERNANCE_GUIDE.md §3	The authoring→resolver field mapping the resolver must apply at load.
@helios/mcp/base.py , @helios/mcp/schemas.py	Once they exist, attach for the semantic.py session.
@mcp_servers.yaml	The registration/config — set registry: ./models/semantic/semantic_layer.yaml .

Prompts to use

Plan + the field-mapping trap first (D9):

```
You are implementing Helios M6 (LEAN: semantic-mcp ONLY). Plan mode – no code yet.
Source of truth: MCP_ARCHITECTURE §6.2/§7/§8/§9. The registry is @models/semantic/semantic_layer.yaml
(v2.0.0). CRITICAL: that file is authored in DESCRIPTIVE field names (metric_name, sql_definition,
aggregation_method, dimensions_supported) but the §9 resolver reads SHORT keys (name, sql, agg, dime
– METRIC_GOVERNANCE_GUIDE §3 defines the mapping. Confirm by opening the file, then propose:
- base.py: typed errors (UnknownMetric -32001, UnknownDimension -32002, DimensionNotPermitted -32003,
InvalidFilter -32004) + normalize_hash(sql);
- semantic.py: load + RI-compile the registry at startup (fail loud per §8 if any dangling ref),
APPLY the §3 field mapping on load, serve get_metric/list_dimensions/build_query per the §9 resol
Tell me where you will apply the metric_name→name mapping and how a ratio composes
SAFE_DIVIDE(SUM(num),SUM(den)) (not AVG of ratios). Show the plan; cite §6.2/§7/§8/§9 per piece.
```

Implement the resolver (copy/adapt the §9 skeleton):

```
[ROLE] Implement Helios M6, file helios/mcp/semantic.py (LEAN: semantic-mcp only).
[SOURCE OF TRUTH] Copy/adapt the build_query resolver in MCP_ARCHITECTURE §9; obey the §7 binding ru
and §8 fail-loud compile. Apply the METRIC_GOVERNANCE_GUIDE §3 field mapping at load
(metric_name→name, sql_definition→sql, aggregation_method→agg, dimensions_supported→dimensions).
[UPSTREAM] Read metric/dimension/grain names ONLY from @models/semantic/semantic_layer.yaml. Errors
from @helios/mcp/base.py.
[TASK] On startup: parse the registry, build metrics/dimensions/grains dicts, run the §8 RI compile
refuse to start on ANY dangling ref. Serve: get_metric(name); list_dimensions(); build_query(metric
dims, filters, window) → governed SQL. _measure(): count/sum → AGG(sql) AS name; ratio →
SAFE_DIVIDE(SUM(num_expr),SUM(den_expr)) AS name (SUM-of-num/SUM-of-den AFTER grouping, NEVER AVG o
per-row ratios); derived → expand_expr(expr) AS name. For countif(reached_*) measures emit the
session-keyed subquery form shown in §6.2. Validate each dim against the metric's dimensions list
```

```
(DimensionNotPermitted) and each name against the registry (UnknownMetric/UnknownDimension). This s
has NO BigQuery client and emits SQL only – it cannot execute.
[TEST-FIRST] Write helios/mcp/tests/test_semantic.py first: (a) UnknownMetric/UnknownDimension → rai
NO SQL string returned; (b) build_query('session_conversion_rate', ['device_category'], window='last_
snapshot-matches the §6.2 governed SQL (session-keyed subquery, SAFE_DIVIDE(COUNTIF(...), COUNT(DIST
session_key))); (c) a copy of the registry with one dangling numerator → server refuses to start;
(d) a non-whitelisted dim → DimensionNotPermitted.
[CONSTRAINTS] The model passes ONLY string names; physical columns live exclusively in registry sql
(I3). Use ONLY names in the attached registry. If a name is missing, STOP and ASK – never fall back
[OUTPUT] Run `pytest helios/mcp/tests/test_semantic.py -v` and paste output. Cite §6.2/§7/§9 per blo
```

Register + smoke-test the server with Claude Code (the grounding demo):

```
[ROLE] Wire semantic-mcp into this Claude Code session and prove I can only query through it.
[TASK] 1) Set mcp_servers.yaml: semantic-mcp transport stdio, command ["python", "-m", "helios.mcp.sem
config.registry ./models/semantic/semantic_layer.yaml (NOT semantic_models.yaml). 2) Register it for
session (show me the exact `claude mcp add` command and the .mcp.json entry):
    claude mcp add semantic-mcp -- python -m helios.mcp.semantic
3) After /mcp shows it connected, smoke-test by calling its tools (not by writing SQL yourself):
- list_dimensions() → confirm the 19 canonical dims;
- get_metric("session_conversion_rate") → confirm numerator=purchasing_sessions, denominator=sess
- build_query("session_conversion_rate", ["device_category"], window="last_28d") → paste the gove
- build_query("session_conversion_rate", ["weather"]) → confirm UnknownDimension (hard fail, no S
[CONSTRAINTS] Do NOT hand-write any SQL. The ONLY way you may obtain a query string is the build_que
If a name is rejected, re-plan against list_dimensions() – do not invent a synonym (G5).
[OUTPUT] Paste the /mcp connection status and each tool call's result.
```

Preventing hallucinations

- The structural guard (I3): `build_query` emits ONLY registry templates, so a hallucinated column (`event_params.foo`) literally cannot appear. Verify Claude did not add a free-SQL escape hatch — there is no path from a string name to a raw column except via the registry `sql` field.
- The §3 field-mapping trap (D2/D3): if the resolver reads `m["sql"]` against the v2 file (which stores `sql_definition`), every measure is empty/KeyError. Make Claude apply the mapping at load and prove it with the §6.2 snapshot test.
- D5/G5: an unknown metric/dimension is a HARD error (`UnknownMetric` / `UnknownDimension`), never a fallback to free SQL. Test the `['weather']` rejection.
- §8 fail-loud (D6): a registry with a dangling ref → the server **refuses to start**. Never serve a half-valid registry. (M5's `validate_semantic.py` is the same compile run standalone — keep both green.)
- Registry filename in `mcp_servers.yaml` MUST be `semantic_layer.yaml`; pointing at `semantic_models.yaml` silently loads the retired 28-metric layer and "works," wrongly.

- Simpson's defense lives in the resolver: ratios are `SAFE_DIVIDE(SUM(num),SUM(den))` after grouping, never `AVG(per_row_ratio)` .

Reviewing generated code

- R1: resolver blocks cite §9; field mapping cites §3; fail-loud cites §8.
- R3: the §6.2 snapshot test ran and matches — the emitted SQL for `session_conversion_rate × device_category` is the session-keyed subquery form with `SAFE_DIVIDE(COUNTIF(purchased),COUNT(DISTINCT session_key))` . This is the contract test that proves grounding.
- R4 (AST/grep check, shared with the eval scorer per §11): parse the emitted SQL; every column/table must be in the registry or GA4 schema — any other name is a hard-zero fail.
- R5: ratios emit `SUM(num)/SUM(den)` after grouping (grep for `AVG(` → must be absent in rate composition); `reached_*` flags compose via the session-keyed subquery so step rates stay ≤ 1 .
- Grounding wiring proof: `/mcp` shows `semantic-mcp` connected, and Claude obtained every query via `build_query` — confirm no hand-written SQL appears in the transcript, and the `['weather']` call returned `UnknownDimension` with NO SQL.
- If you instead built `warehouse-mcp` (the other half), additionally verify R6: `run_query` without a prior `dry_run` → `NotDryRunFirst` ; `max_bytes_billed` capped at 5 GiB → `ByteBudgetExceeded` on over-scan; `reconcile` within 0.5% of a hand control query; a write SQL → error (read-only SA). The hash must normalize identically (whitespace-collapsed, lowercased) in `dry_run` and `run_query` or the gate spuriously fails.

M6b - stats / experiment / report MCP [Wk2-v1: the chosen server only; rest v2]

Full design = three data-independent servers: `stats-mcp` (only path to math, seeded), `experiment-mcp` (powered test cards), `report-mcp` core (`render_brief` / `export`). **LEAN: build the ONE server you chose.** If you built `semantic-mcp` in M6, the natural M6b pick is `stats-mcp` for `decompose_change` — the thesis centerpiece that dissolves Simpson's paradox by splitting ΔR into mix/rate/interaction. It **fails silently** (wrong numbers, not an error), so it gets a GOLDEN test FIRST. `experiment-mcp` and `report-mcp` memory are **[v2-later]**; a one-line "recommended next test" in the brief replaces `design_experiment` for the lean cut (LEAN_SCOPE).

Context to load

- CLAUDE.md (auto: the decomposition identity verbatim, `rng_seed: 1729`, G2 "never compute a statistic in prose").
- `@docs/MCP_ARCHITECTURE.md` — §6.3 (stats tool I/O), §9 (the `decompose_change` skeleton — copy verbatim), §11 (the GOLDEN test: `mix=-0.0018`, `rate=0`, `interaction=0`, `delta_R=-0.0018 ±1e-9`), §5 (errors: `SegmentMismatch`, `InsufficientData`).
- `@models/semantic/semantic_layer.yaml` — only if you also wire `experiment-mcp.design_experiment` (it validates target/guardrail metrics against the registry); otherwise not needed (stats has no data access).

Files to provide

File	Why
<code>@docs/MCP_ARCHITECTURE.md</code> (§6.3, §9, §11, §5)	<code>decompose_change</code> skeleton + the golden values; copy verbatim.
<code>@helios/mcp/base.py</code>	Typed errors (<code>SegmentMismatch</code> -32031, <code>InsufficientData</code> -32030, <code>ZeroSample</code> -32032).
<code>@helios/mcp/stats.py</code>	Once it exists, for iteration.
<code>@models/semantic/semantic_layer.yaml</code>	Only if building <code>experiment-mcp</code> (registry validation of test-card metrics).

Prompts to use

Golden test FIRST, then the implementation (TDD; the silent-failure guard):

```
[ROLE] Implement Helios M6b, file helios/mcp/stats.py (LEAN: stats-mcp; decompose_change is the keys
[SOURCE OF TRUTH] Copy the decompose_change skeleton from MCP_ARCHITECTURE §9 VERBATIM – it implemen
FOUNDATION identity exactly:  $mix = \sum \Delta w \cdot r(t_0)$ ,  $rate = \sum w(t_0) \cdot \Delta r$ ,  $interaction = \sum \Delta w \cdot \Delta r$ ,
 $\Delta R = mix + rate + interaction$ . Errors from @helios/mcp/base.py. Seed rng_seed=1729 at import.
[TEST-FIRST] BEFORE the code, write helios/mcp/tests/test_decompose_golden.py with the §11 golden ca
    t0 = [{"seg": "desktop", "w": 0.40, "r": 0.030}, {"seg": "mobile", "w": 0.60, "r": 0.012}]
    t1 = [{"seg": "desktop", "w": 0.30, "r": 0.030}, {"seg": "mobile", "w": 0.70, "r": 0.012}] # weights shift
    assert mix_effect == -0.0018, rate_effect == 0, interaction == 0, delta_R == -0.0018 (all ±1e-9)
Also: identity  $mix+rate+interaction == delta\_R$  on 100 random seeded inputs; SegmentMismatch when t0
segment sets differ; determinism (same seed → byte-identical output).
[TASK] Implement decompose_change(metric, dim, t0, t1) per §9 (raise SegmentMismatch on differing se
sets). Add significance_test(a,b,kind="proportion"|"mean") matching a scipy reference. (detect_anom
forecast/cohort_retention/rfm_segment are v2 – stub or omit per LEAN.) NO data access; NO warehouse
[CONSTRAINTS] mix uses r(t0); rate uses w(t0). Do NOT swap the t0/t1 baselines – it flips attributio
golden test is the only thing that catches it. Numbers are tool outputs; the LLM never does this ar
[OUTPUT] Run `pytest helios/mcp/tests/ -v` and paste output (golden + identity + determinism must pa
Cite §9/§11.
```


Register + smoke-test stats-mcp with Claude Code:

```
[ROLE] Wire stats-mcp into Claude Code and prove the decomposition is a tool, not prose.
[TASK] 1) mcp_servers.yaml: stats-mcp transport stdio, command ["python","-m","helios.mcp.stats"],
      config.rng_seed 1729. 2) Register for this session:
      claude mcp add stats-mcp -- python -m helios.mcp.stats
3) After /mcp shows connected, call decompose_change with the §11 golden inputs (desktop/mobile,
   weights 0.40/0.60 → 0.30/0.70, rates 0.030/0.012 unchanged). Paste the result and confirm
   mix_effect=-0.0018, rate=0, interaction=0. 4) Re-run identical input twice → identical output (seed
[CONSTRAINTS] Do NOT compute mix/rate/interaction yourself in text (G2). The ONLY source of the num
the tool call. State in one line what this result MEANS: the blended rate fell purely because traff
shifted to lower-converting mobile – mix-shift, not behavior change.
[OUTPUT] Paste /mcp status, the tool result, and the two-run determinism check.
```

(If `experiment-mcp` is your chosen server instead, add: `design_experiment(hypothesis, metric)` must validate `metric` and every guardrail metric against `@models/semantic/semantic_layer.yaml` — a card can only target a governed metric like `net_revenue`, never `refund_rate`; test that an ungoverned name → `UnknownMetric`.)

Preventing hallucinations

- G2 specialized: every statistic is a `stats-mcp` output, never prose. In the smoke test, forbid Claude from hand-computing mix/rate in text — the numbers must come from the tool call.
- The silent-failure guard (D6): `decompose_change` returns wrong numbers, not an error, when the formula is off. The §11 golden test (`mix=-0.0018`) is the ONLY thing that catches a swapped `t0 / t1` baseline — write it FIRST and make it pass before trusting any output.
- Copy the §9 skeleton verbatim: `mix` uses `r(t0)`, `rate` uses `w(t0)`. Adapting the algebra from memory is the classic drift here.
- Determinism (seed 1729): seed at import so the byte-identical test passes and eval grading is reproducible. Non-determinism breaks both.
- `SegmentMismatch` is not ignorable: if `t0 / t1` segment sets differ the decomposition is undefined (mix double-counts appearing/disappearing segments) — raise, don't paper over.
- `report-mcp` (if built) gets NO BigQuery client — rendering is pure string work; `render_brief([])` → `EmptyFindings` (emit a "no material change" brief, never fabricate a finding). Memory tools (`save_diagnosis` / `recall_prior`) are [v2-later], not M6b.

Reviewing generated code

- R3 (the keystone check): the GOLDEN test ran and passed exactly — `mix_effect=-0.0018`, `rate_effect=0`, `interaction=0`, `delta_R=-0.0018` ($\pm 1e-9$) on the §11 example. No PR without this passing.

- R1: `decompose_change` cites §9; the golden values cite §11.
- R5: verify `mix` uses `r(t0)` and `rate` uses `w(t0)` by READING the diff against the §9 formulas — a swapped baseline still satisfies `mix+rate+interaction == delta_R` (the identity check passes!) yet attributes wrongly; only the golden test distinguishes them.
- R2: determinism — re-run any stats tool on the same input twice; outputs byte-identical (seed 1729). `significance_test` matches a scipy reference within tolerance.
- Wiring proof: `/mcp` shows `stats-mcp` connected; the golden result came from the tool call (in the transcript), not Claude's arithmetic.
- For `experiment-mcp` (if chosen): R4 — `design_experiment` rejects an ungoverned metric (`UnknownMetric`); `power_analysis` two-proportion sample size matches the closed-form reference. For `report-mcp`: R6-style — `render_brief([])` → `EmptyFindings`, and every number in a brief traces to a backing tool-output hash (faithfulness rule, §11).